



# طراحی واحدهای پردازش بیتی پیشرفته معماری کامپیوتر - گزارش کار تمرین عملی دوم

استاد  
دکتر سربازی آزاد

دانشگاه صنعتی شریف  
بهار 1405

محمد مهدی مرادی - 403106681

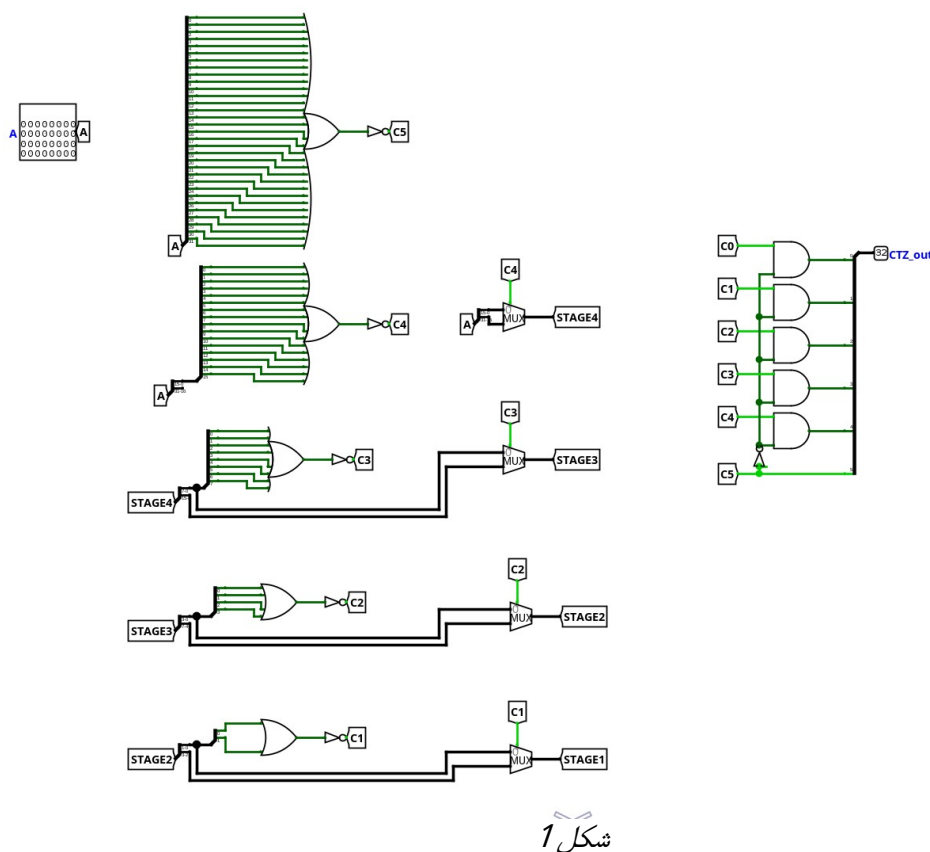
# فهرست

|        |                             |
|--------|-----------------------------|
| 3..... | مقدمه                       |
| 3..... | 1-طراحی مدار شمارشگر        |
| 6..... | 2-طراحی مدار Barrel Shifter |

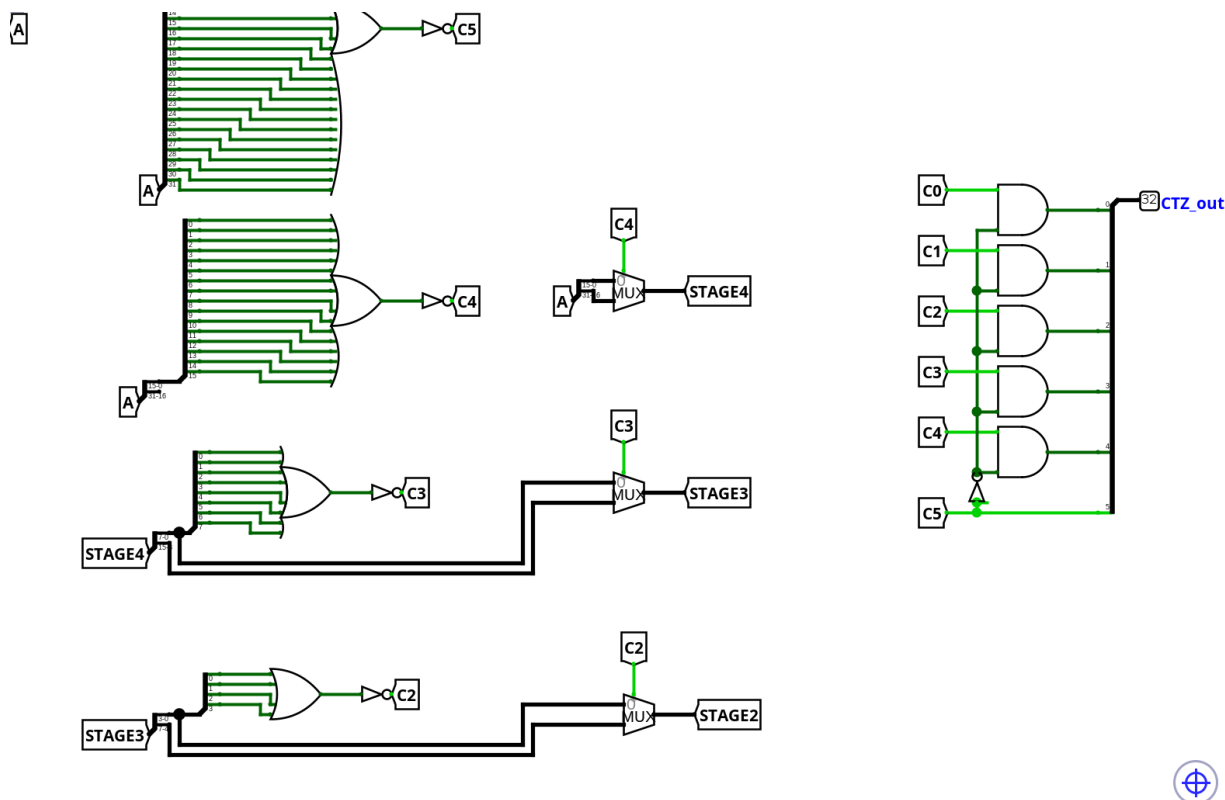
هدف از این آزمایش، طراحی و پیاده‌سازی دو بخش مهم از واحد محاسبه و منطق (ALU) در پردازنده‌ها با استفاده از نرم‌افزار Logisim است. این دو بخش شامل یک مدار شمارشگر (CTO, CLO, CTZ, CLZ) و یک شیفت‌دهنده بشکهای (Barrel Shifter) ۳۲ بیتی برای انجام عملیات شیفت به چپ و راست (منطقی و حسابی) و می‌باشد. در این طراحی‌ها، تمرکز اصلی بر روی بهینه‌سازی سرعت و دستیابی به تاخیر لگاریتمی بوده است تا عملکرد مدار برای پردازنده‌های واقعی مناسب باشد.

## ۱- طراحی مدار شمارشگر

برای طراحی مدار شمارشگر ابتدا مدار CTZ را طراحی می‌کنیم. این مدار وظیفه دارد تعداد صفرهای متوالی را از سمت راست (کم‌ارزش‌ترین بیت) تا رسیدن به اولین بیت ۱<sup>۱</sup> بشمارد. برای این کار از روش جستجوی باینری (Binary Search) در سطح سخت‌افزار استفاده شده است تا تاخیر به حداقل برسد.



در شکل یک، ساختار درختی مدار مشخص است. مدار ابتدا داده ۳۲ بیتی را به دو نیمه ۱۶ بیتی تقسیم می‌کند و بررسی می‌کند که آیا نیمه سمت راست تماماً صفر است یا خیر.



شکل 2

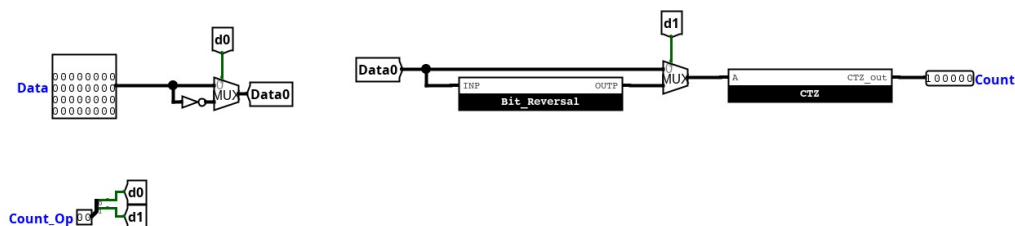
همان‌طور که در این شکل ۲ می‌کنید، برای تشخیص صفر بودن یک بلوک از بیت‌ها، از گیت‌های NOR استفاده شده است. اگر خروجی NOR یک شود (یعنی تمام بیت‌های آن بخش صفر باشند)، مالتی‌پلکسر به ما دستور می‌دهد که باید در نیمه با ارزش‌تر به دنبال اولین '۱' بگردیم و به تعداد بیت‌های این نیمه، به خروجی نهایی اضافه می‌شود.

در سمت راست شکل ۲ می‌بینیم که  $\text{not } C_5$  با بقیه  $C$  ها and شده، به این دلیل است که اگر این بیت یک بود یعنی تمام ۳۲ بیت ورودی ما صفر است و فقط باید این سیگنال یک شود تا ماکسیمم عدد ۳۲ را داشته باشیم.

حالا برای اینکه بتوانیم شمارش‌های دیگر (CTO, CLZ, CLO) را داشته باشیم با این زیر مدار CTZ می‌ایم همه را می‌سازیم. اینگونه که اگر بخواهیم صفرهای سمت پر ارزش را بسازیم با استفاده از زیر مدار Bit Reversal که مدار آن را در بخش بعدی توضیح می‌دهیم، بیت‌های ورودی را برعکس می‌کنیم و اینگونه شمردن بیت‌های صفر سمت پر ارزش معادل میشود با شمردن بیت‌های صفر سمت کم ارزش وقتی که ورودی برعکس شده است. برای شمردن یک‌ها، چه سمت کم ارزش و چه سمت پر ارزش می‌ایم و بیت‌ها را نات می‌کنیم، اینگونه باید برای شمردن بیت‌های یک در ورودی اصلی، بیاییم و بیت‌های صفر را در ورودی نات شده بشماریم. حالا اگر بخواهیم بیت‌های یک سمت کم ارزش را بشماریم که با استفاده از همان زیر مدار CTZ می‌شماریم و اگر بخواهیم بیت‌های یک سمت پر ارزش را بشماریم با استفاده از راهکار قبلی، می‌ایم و ورودی را برعکس می‌کنیم و بیت‌های یک را می‌شماریم.

برای مشخص کردن اینکه در کدام حالت هستیم نیز از یک مولتی‌پلکسر استفاده می‌کنیم.

مدار نهایی را در شکل ۳ مشاهده می‌کنید



شکل 3

```
logisim workspace : /home/mahdi/logisim_evolution_workspace
inpins : ['Count_Op', 'Data']
outpins : ['Count']
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Labeled main/CTZ_1
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Labeled main/Bit_Reversal_1
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "CTZ"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "CTZ" has 11 nets and 6 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "CTZ" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "Bit_Reversal"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "Bit_Reversal" has 32 nets and 2 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "Bit_Reversal" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "main"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" has 8 nets and 7 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - The Board ALCHITRY_AU_IO has:
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 6 Button(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 3 DIPSwitch(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 32 Led(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 1 SevenSegmentScanning(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/plexers/Multiplexer_bus_2.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/AND_GATE.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/OR_GATE.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/OR_GATE_4_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/OR_GATE_8_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/OR_GATE_32_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/gates/OR_GATE_16_INPUTS.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/circuit/CTZ.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/circuit/Bit_Reversal.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/verilog/toplevel/logisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/scripts/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/2.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

HW2/counter-tb.v /home/mahdi/logisim_evolution_workspace/2.circ.tmp
HW2/counter-tb.v:75: Warning: Calling system function $random() as a task.
HW2/counter-tb.v:75: The functions return value will be ignored.
CTZ
CT0
CLZ
CLO
ACCEPTED
280 / 280
```

شکل 4

در شکل ۴، موفقیت‌آمیز بودن نتایج تست‌بنچ برای مدار شمارشگر قابل مشاهده است.

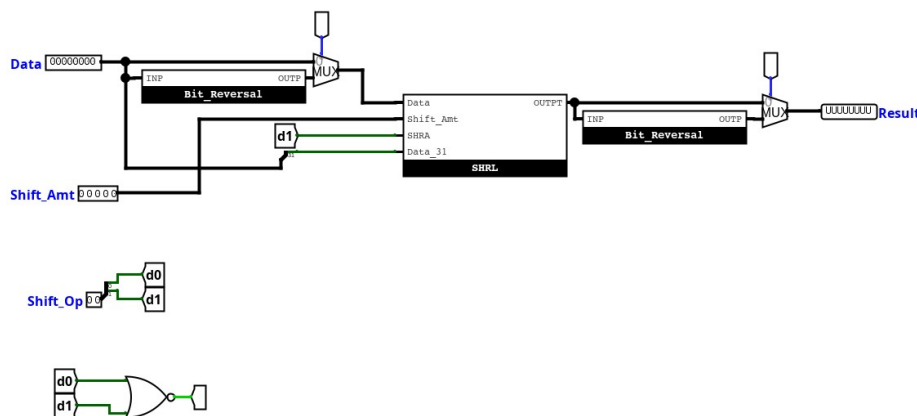
### محاسبه مرتبه تاخیر مدار شمارشگر:

استفاده از رویکرد جستجوی باینری باعث می‌شود تا در هر مرحله، طول رشته مورد بررسی نصف شود. برای یک کلمه  $N$  بیتی، ما به

$\log_2(N)$  مرحله نیاز داریم (برای ۳۲ بیت، ۵ مرحله). در هر مرحله، سیگنال‌ها از گیت‌های NOR و مالتی‌پلکسرها عبور می‌کنند که هر کدام تاخیر ثابتی دارند. با صرف نظر کردن از ضرایب ثابت، مرتبه تاخیر این مدار نیز برابر با  $O(\log(N))$  محاسبه می‌شود که در مقایسه با روش‌های شمارش خطی با تاخیر  $O(N)$ ، بسیار سریع‌تر و کارآمدتر است.

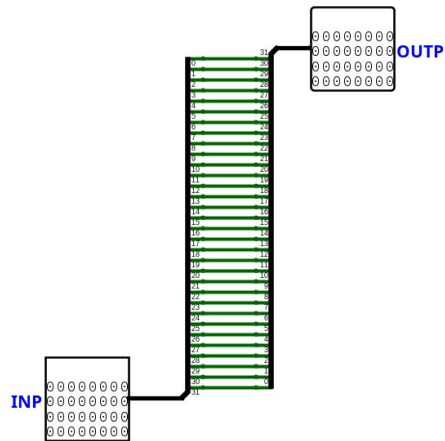
## ۲- طراحی مدار Barrel Shifter

در این بخش یک شیفت‌دهنده ۳۲ بیتی طراحی شده است که می‌تواند سه عملیات Shift Left Logical (SLL)، Shift Right Arithmetic (SRA) و Right Logical (SRL) را انجام دهد. به جای استفاده از سه شیفت‌دهنده مجزا، از ایده معکوس کردن بیت‌ها (Bit Reversal) استفاده شده است تا با یک مدار شیفت به راست، بتوان شیفت به چپ را نیز پیاده‌سازی کرد.



شکل 5

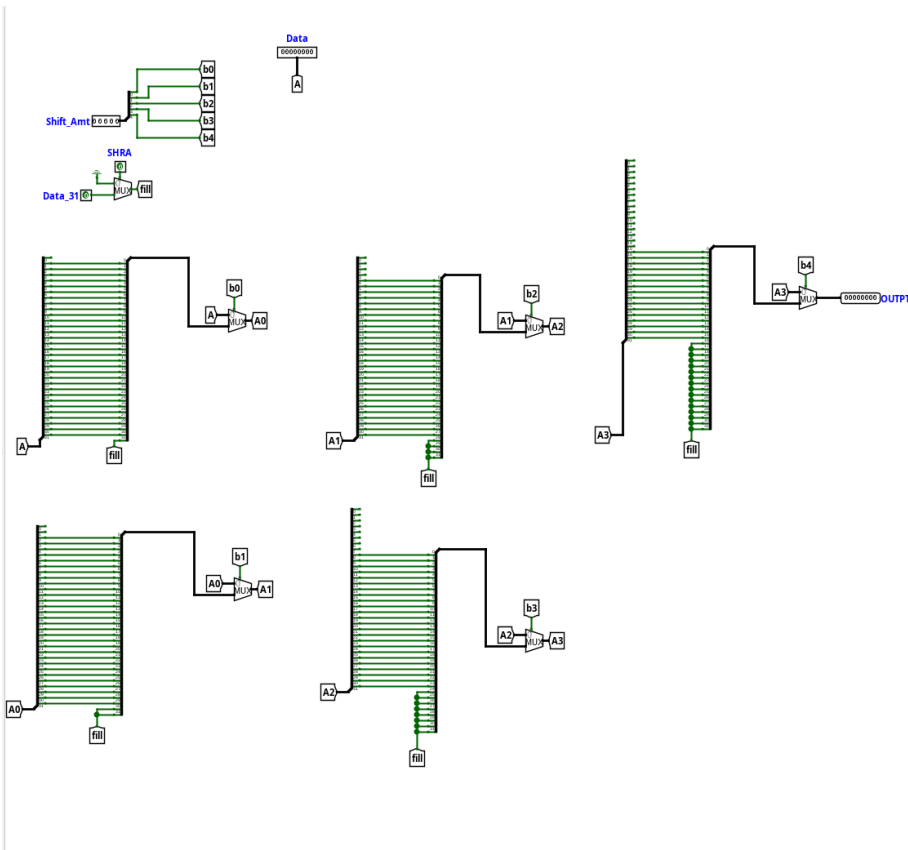
همان‌طور که در شکل ۵ مشاهده می‌کنید، مسیر داده‌ها بر اساس نوع عملیات کنترل می‌شود. اگر عملیات شیفت به چپ (SLL) باشد، داده‌های ورودی ابتدا توسط بلوک Bit Reversal معکوس می‌شوند، سپس وارد شیفت‌دهنده به راست شده و خروجی مجدداً معکوس می‌شود.



شکل 6

در شکل ۶، ساختار داخلی بلوک معکوس کننده را می بینید. این بلوک هیچ گیت منطقی فعالی ندارد و تنها با استفاده از Splitterها، ترتیب سیم ها را از بیت ۰ تا ۳۱ به ۳۱ تا ۰ تغییر می دهد.

برای انجام شیفت به راست، از یک زیرمدار به نام SHRL استفاده شده است که به صورت آبشاری با استفاده از مالتی پلکسرها و اسپلیترها، شیفت های ۱، ۲، ۴، ۸ و ۱۶ بیتی را انجام می دهد.

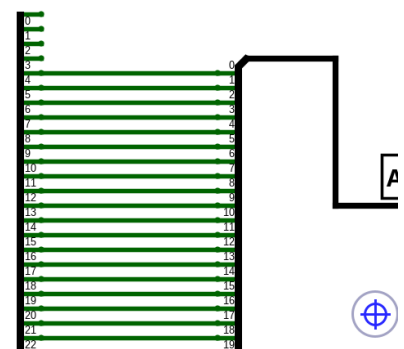
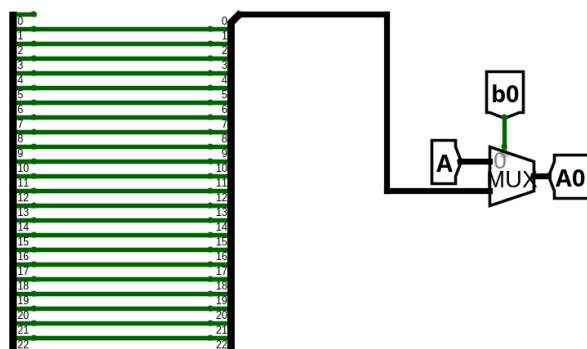
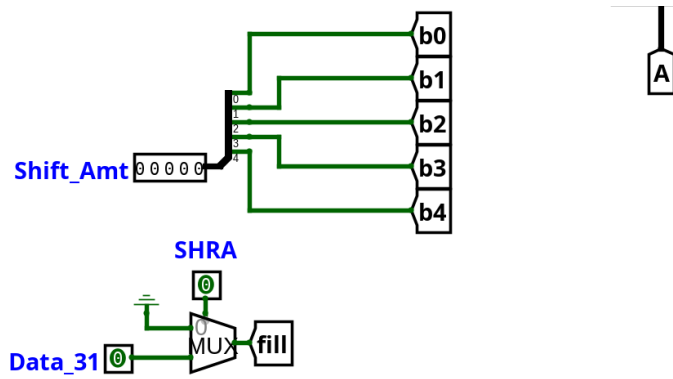


شکل ۷

در شکل ۷، نحوه اتصال اسپلیترها برای دور انداختن بیت های سمت راست و وارد کردن بیت های جدید از سمت چپ (بیت با ارزش) مشخص است.

یکی از چالش های اصلی، انتخاب بیت پرکننده (Fill Bit) برای شیفت به راست است. در شیفت منطقی (SRL) این بیت باید صفر باشد، اما در شیفت حسابی (SRA) باید برابر با بیت علامت (بیت ۳۱ داده اصلی) باشد.





شکل 8

همان‌طور که در شکل ۸ مشخص است، با استفاده از سیگنال کنترلی، مشخص می‌شود که بیت‌های ورودی جدید از نوع صفر باشند یا متصل به بیت ۳۱ (Sign bit).

```

logisim workspace : /home/mahdi/logisim_evolution_workspace
inpins : ['Data', 'Shift_Op', 'Shift_Amt']
outpins : ['Result']
[main] INFO com.cburch.logisim.gui.generic.OptionPane - "SHRL" : One component in the circuit has the same label as the component name of the newly placed component. This label has been removed!
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Labeled main/Bit Reversal_1
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Labeled main/SHRL_1
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Labeled main/Bit Reversal_2
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "Bit Reversal"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "Bit Reversal" has 32 nets and 2 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "Bit Reversal" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "SHRL"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "SHRL" has 133 nets and 12 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "SHRL" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Building netlist for sheet "main"
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" has 1 nets and 8 buses.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Circuit "main" passed DRC check.
[main] INFO com.cburch.logisim.fpga.gui.Reporter - The Board ALCHITRY_AU_IO has:
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 6 Button(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 3 DIPSwitch(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 32 Led(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - 1 SevenSegmentScanning(s)
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/gates/NOR_GATE.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/plexers/Multiplexer_bus_2.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/circuit/Bit_Reversal.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/plexers/Multiplexer_2.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/circuit/SHRL.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/circuit/main.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating HDL file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/verilog/toplevel/logisimTopLevelShell.v
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/scripts/vivadoCreateProject.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/scripts/vivadoConstraints.xdc
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/scripts/vivadoGenerateBitStream.tcl
[main] INFO com.cburch.logisim.fpga.gui.Reporter - Creating script file: /home/mahdi/logisim_evolution_workspace/3.circ.tmp/main/scripts/vivadoLoadBitStream.tcl

HW2/shifter-tb.v /home/mahdi/logisim_evolution_workspace/3.circ.tmp
HW2/shifter-tb.v:25: Warning: Calling system function $random() as a task.
HW2/shifter-tb.v:25: The functions return value will be ignored.
ALL
SRL
SRA
ACCEPTED
150 / 150

```

## شکل 9

شکل ۹ نشان‌دهنده اجرای موفقیت‌آمیز تست‌بنچ‌ها و تایید صحت عملکرد مدار شیفت‌دهنده در تمامی حالت‌ها است.

## محاسبه مرتبه تاخیر شیفت‌دهنده:

در این طراحی، به جای استفاده از شیفت‌دهنده‌های ترتیبی که تاخیر  $O(N)$  دارند، از ساختار درختی و آبشاری (Cascade) استفاده شده است. مدار برای شیفت یک عدد  $N$  بیتی، به  $\log_2(N)$  طبقه مالتی‌پلکسر نیاز دارد (برای ۳۲ بیت، ۵ طبقه). از آنجایی که بلوک‌های Bit Reversal تنها شامل سیم‌کشی هستند، تاخیر آن‌ها صفر است. تاخیر هر مالتی‌پلکسر نیز یک مقدار ثابت (Constant) در نظر گرفته می‌شود. بنابراین تاخیر کل مدار متناسب با تعداد طبقات MUX است و مرتبه تاخیر آن برابر با  $O(\log(N))$  می‌باشد که بهینه‌ترین حالت ممکن است.